



PROGRAMMING MODULE ASSIGNMENT



Level 4 – CY Batch 2

Table of Contents

OVERVIEW OF ASSIGNMENT.....	2
TASKS.....	3
TASK 1 – Print an American flag.....	3
EXPLANATION OF THE CODE.....	4
PSEUDO CODE	5
SOURCE CODE	7
SCREENSHOT OF THE CODE	10
OUTPUT OF THE CODE	12
TASK 2 – Calculate Net Salary	13
EXPLANATION OF THE CODE.....	14
PSEUDO CODE	15
SOURCE CODE	16
SCREENSHOT OF THE CODE	18
OUTPUT OF THE CODE	19
TASK 3 - Online Bookshop.....	20
EXPLANATION OF THE CODE.....	21
FLOWCHART	22
SOURCE CODE	23
OUTPUT OF THE CODE	29
TASK 4 - Favorite Destination.....	31
EXPLANATION OF THE CODE.....	32
FLOWCHART	33
SOURCE CODE	34
SCREENSHOT OF THE CODE	39
OUTPUT OF THE CODE	42
TASK 5 - Account Management	43
EXPLANATION OF THE CODE.....	44
UML DIAGRAM	45
SOURCE CODE	46
SCREENSHOT OF THE CODE	50
OUTPUT OF THE CODE	52

OVERVIEW OF ASSIGNMENT

This assignment focuses on signifying detailed understanding of java programming based on five tasks. Each task is based on a specific programming concept such as calculation, user input, arrays and object oriented programming. This submission consists of written codes, pseudo codes or flow-charts or UML diagram, explanation of each code and program outputs that demonstrates problem solving abilities and coding application in Java.

Task Overview

Task 1 – Print an American flag

- Program to display an American flag with stars, stripes, stick and stairs.
- Loops for efficient pattern display, to differentiate alternate lines and parts of the flag.

Task 2 – Calculate Net salary

- Program to calculate an employee's net salary on the basis of their basic salary.
- Perform calculation for HRA, DA, Gross Salary and Income tax.

Task 3 – Online Bookshop

- Program to run an online bookstore
- Allows users to select any choice of books and quantities and displaying total cost in the end.

Task 4 – Favorite Destination

- Program to survey users about their favorite city and display the most voted city.
- Using ArrayLists to count votes and cities.

Task 5 – Account Management (OOP)

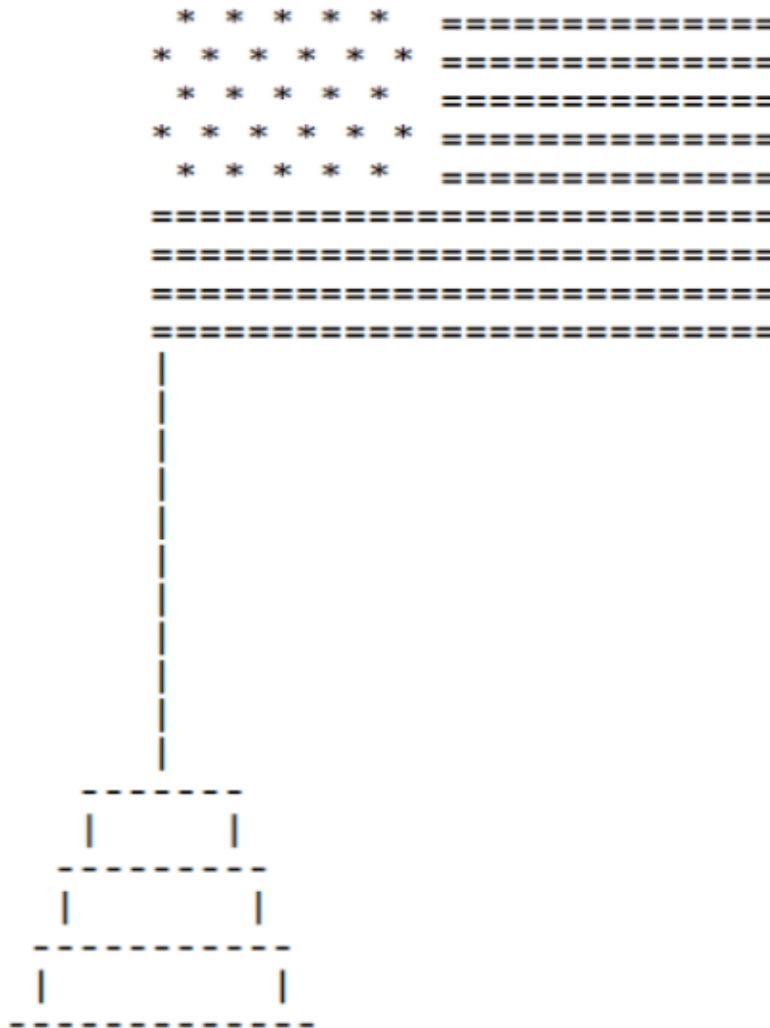
- Program to create class Account and include account related details.
- Implement methods like deposit, withdrawal and interest rate calculation.

TASKS

TASK 1 – Print an American flag

Write a Java program to print an American flag on the screen.

Expected Output:



EXPLANATION OF THE CODE

This program prints an American flag with stars, stripes, a stick, and stairs below it. The program is divided into different parts to create each part of the flag.

- **Stars and Stripes:**

- To print alternate lines of stars (*) and stripes (=), a for loop executes five times.

- Even lines (0, 2, 4): Prints " * * * * * =====".
- Odd lines (1, 3): Prints " * * * * * * =====".

- **Stripes:**

- To print the stripes, a for loop is executed four times using " =====".

- **Stick:**

- To print the stick a for loop is executed seven times using "|".

- **Stairs:**

- Nested for loops iterate to form a stairs.
 - The outer loop (m) runs through rows.
 - The inner loop (n) runs through columns, printing dashes (-) or spaces for even rows and vertical bars (|) or spaces for odd rows.
- The width of the stairs increments after odd rows.

PSEUDO CODE

BEGIN

// Flag with stars and stripes

FOR i **FROM** 0 **TO** 4

IF the remainder **WHEN** i is divided by 2 is 0 **THEN**

PRINT five stars and equals

ELSE

PRINT six stars and equals

END IF

END FOR

// Stripes below the stars

FOR i **FROM** 0 **TO** 3

PRINT equals only

END FOR

// Stick below the flag

SET stickheight **TO** 7

FOR i **FROM** 0 **TO** 6

PRINT vertical bar with indentation

END FOR

// Stairs below the stick

SET rows **TO** 7

SET width **TO** 3

FOR m **FROM** 0 **TO** 6

```
FOR n FROM 0 TO 39
    IF the remainder WHEN m is divided by 2 is 0 THEN
        IF n lies between (6-width) AND (6+width) THEN
            PRINT a dash
        ELSE
            PRINT space
        END IF
    ELSE
        IF n is equal to (6 - width) OR (6 + width) THEN
            PRINT vertical bar
        ELSE
            PRINT space
        END IF
    END IF
END FOR
IF the remainder WHEN m is divided by 2 is not 0 THEN
    Increase width BY 1
END IF
PRINT new line
END FOR
END
```

SOURCE CODE

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */

// Task - To print an American flag.

public class Flag {
    public static void main(String[] args) {

        // Flag with stars and stripes
        // Declare and initialize i
        for (int i = 0; i < 5; i++) {

            // print line 0, 2 & 4 of flag
            if (i % 2 == 0) {
                System.out.println("          * * * * *
=====");

            } // print line 1, 3 & 5 of flag
            else {
                System.out.println("          * * * * *
=====");
            }
        }
    }
}
```



```

// Stripes below the stars
// print stripes four times
for (int i = 0; i < 4; i++) {
    System.out.println("
=====");
}

// Stick below the flag
int stickHeight = 7;
// print vertical bar six times
for (int i = 0; i < stickHeight; i++) {
    System.out.println("      |");
}

// Stairs below the stick
int rows = 7;
int width = 3;

// loop through each row
for (int m = 0; m < rows; m++) {
    // loop through each column
    for (int n = 0; n < 40; n++) {
        // if row is even
        if (m % 2 == 0) {
            // print a dash
            if (n >= 6 - width && n <= 6 + width) {
                System.out.print("-");
            } // print space

```

```

        else {
            System.out.print(" ");
        }
    } // if row is odd
else {
    // print vertical bar at the edges
    if (n == 6 - width || n == 6 + width) {
        System.out.print("|");
    } // print space
    else {
        System.out.print(" ");
    }
}

// Expand the stairs
// Increment width when mod not equal to 0
if (m % 2 != 0) {
    width++;
}

// move to next line
System.out.print("\n");
}

}

}

```

SCREENSHOT OF THE CODE

The program is written in Apache NetBeans IDE 23

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */

// Task - To print an American flag.

public class Flag {
    public static void main(String[] args) {

        // Flag with stars and stripes
        // Declare and initialize i
        for (int i = 0; i < 5; i++) {

            // print line 0, 2 & 4 of flag
            if (i % 2 == 0) {
                System.out.println("      * * * * * =====");

            } // print line 1, 3 & 5 of flag
            else {
                System.out.println("      * * * * * =====");
            }
        }

        // Stripes below the stars
        // print stripes four times
    }
}
```

Fig 1.1 Source code for task 1

```
        for (int i = 0; i < 4; i++) {
            System.out.println("      =====");
        }

        // Stick below the flag
        int stickHeight = 7;
        // print vertical bar six times
        for (int i = 0; i < stickHeight; i++) {
            System.out.println("      |");
        }

        // Stairs below the stick
        int rows = 7;
        int width = 3;

        // loop through each row
        for (int m = 0; m < rows; m++) {
            // loop through each column
            for (int n = 0; n < 40; n++) {
                // if row is even
                if (m % 2 == 0) {
                    // print a dash
                    if (n >= 6 - width && n <= 6 + width) {
                        System.out.print("-");
                    } // print space
                } else {
                    System.out.print(" ");
                }
            }
        }
    }
}
```

Fig 1.2 Source code for task 1

```

        else {
            System.out.print(" ");
        }
    } // if row is odd
    else {
        // print vertical bar at the edges
        if (n == 6 - width || n == 6 + width) {
            System.out.print("|");
        } // print space
        else {
            System.out.print(" ");
        }
    }
}

// Expand the stairs
// Increment width when mod not equal to 0
if (m % 2 != 0) {
    width++;
}
// move to next line
System.out.print("\n");
}
}

```

Fig 1.3 Source code for task 1

OUTPUT OF THE CODE

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---
  * * * * * =====
* * * * * * =====
  * * * * * =====
* * * * * * =====
  * * * * * =====
=====
=====
=====
=====
|
|
|
|
|
|
|
|
-----
|      |
-----
|      |
-----
|      |
-----
|      |
-----
-----
BUILD SUCCESS
-----
Total time:  5.442 s
Finished at: 2024-12-11T14:24:46+04:00
-----
```

Fig 1.4 Output for task 1

TASK 2 – Calculate Net Salary

Write a Java program to compute the net salary of an employee by considering the parameters called HRA (House rent allowance), DA (Dearness allowance), GS (Gross salary) and income tax considering following parameters. $HRA = 15\%$ of basic salary, $DA = 75\%$ of basic salary, $GS = \text{basic salary} + DA + HRA$, Income tax = 35% of gross salary. Net salary = $GS - \text{income tax}$

EXPLANATION OF THE CODE

This program calculates Net Salary of employee based on their basic salary. It begins by asking user to enter their salary. If the salary is less than or equal to 0, it prints Invalid input. Else the following calculation are done:

- **HRA (House Rent Allowance)** - is calculated as 15% of the salary.
- **DA (Dearness Allowance)** is calculated as 75% of the salary.
- **Gross Salary** is the sum of the salary, HRA and DA.
- **Income Tax** is calculated as 35% of the Gross Salary.
- **Net Salary** is difference between Income Tax and the Gross Salary.

The program then displays the calculated amounts (salary, HRA, DA, gross salary, Income tax and net salary).

PSEUDO CODE

BEGIN

PRINT "Enter the salary:"

INPUT salary

IF salary <=0 **THEN**

PRINT "Invalid input"

ELSE

SET HRA = 15% salary

SET DA = 75% salary

SET GS = basic salary+DA+HRA

SET = incomeTax 35% gross salary

SET netSalary = GS – incomeTax

PRINT "Basic Salary:" salary

PRINT "HRA:", HRA

PRINT "DA:", DA

PRINT "GS:", GS

PRINT "Income Tax:", incomeTax

PRINT "Net Salary:", netSalary

END IF

END

SOURCE CODE

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */

import java.util.Scanner;

public class NetSalary {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        // User must enter their basic salary
        System.out.print("Enter the salary: ");
        double salary = input.nextDouble();

        // check if salary is less than or equal to 0
        if(salary<=0){
            // Invalid salary
            System.out.println("Invalid input");
        }
        else{
            // Calculate House rent allowance
            double HRA = 0.15 * salary;

            // Calculate Dearness allowance
            double DA = 0.75 * salary;
```

```
// Calculate Gross salary
double GS = salary + HRA + DA;

// Income Tax
double incomeTax = 0.35 * GS;

// Calculate Net Salary
double netSalary = GS - incomeTax;

// Print the calculated values
System.out.println("Basic Salary: " + salary);
System.out.println("HRA: " + HRA);
System.out.println("DA: " + DA);
System.out.println("Gross Salary: " + GS);
System.out.println("Income Tax: " + incomeTax);
System.out.println("Net Salary: " + netSalary);
}
}
}
```

SCREENSHOT OF THE CODE

The program is written in Apache NetBeans IDE 23

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */

import java.util.Scanner;
public class NetSalary {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        // User must enter their basic salary
        System.out.print("Enter the salary: ");
        double salary = input.nextDouble();

        // check if salary is less than or equal to 0
        if(salary<=0){
            // Invalid salary
            System.out.println("Invalid input");
        }
        else{
            // Calculate House rent allowance
            double HRA = 0.15 * salary;

            // Calculate Dearness allowance
            double DA = 0.75 * salary;
```

Fig 2.1 Source code for task 2

```
        // Calculate Gross salary
        double GS = salary + HRA + DA;

        // Income Tax
        double incomeTax = 0.35 * GS;

        // Calculate Net Salary
        double netSalary = GS - incomeTax;

        // Print the calculated values
        System.out.println("Basic Salary: " + salary);
        System.out.println("HRA: " + HRA);
        System.out.println("DA: " + DA);
        System.out.println("Gross Salary: " + GS);
        System.out.println("Income Tax: " + incomeTax);
        System.out.println("Net Salary: " + netSalary);
    }
}
```

Fig 2.2 Source code for task 2

OUTPUT OF THE CODE

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---  
Enter the salary: 10,000  
Basic Salary: 10000.0  
HRA: 1500.0  
DA: 7500.0  
Gross Salary: 19000.0  
Income Tax: 6650.0  
Net Salary: 12350.0  
-----  
BUILD SUCCESS  
-----  
Total time: 33.845 s  
Finished at: 2024-12-12T12:49:31+04:00
```

Fig 2.3 Output for task 2

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---  
Enter the salary: -100  
Invalid input  
-----  
BUILD SUCCESS  
-----  
Total time: 12.960 s  
Finished at: 2024-12-19T21:38:52+04:00  
-----
```

Fig 2.4 Output for task 2

TASK 3 - Online Bookshop

Write a Java program to simulate an online Book Shop. The program should begin by displaying a list of books and their prices. There should be a minimum of 10 books offered. The program should ask the user to choose a book and then ask the user to enter the quantity they require of that product. The program should then allow the user to keep choosing more products and quantities until they enter something to indicate they want to end the program (e.g. a given number or 'e' or 'exit'). The program should then tell the user the total amount for the products they have selected.

EXPLANATION OF THE CODE

This program is an online bookshop where users can go through a list of books and select a book they wish to buy by specifying the quantity. It initializes a list of 10 books with its prices then the user is provided with these options:

Methods -

- **booknames()**: Displays all available books and their prices.
- **bookchoice()**:the user has to choose a book by number.
- **bookquantity()**:the user has to enter the quantity of the selected book.

After each book is chosen the total cost is updated. The program runs until the user chooses to exit by entering 0. Once the user exits the program displays the total cost of the books purchased with a thank you message.

FLOWCHART

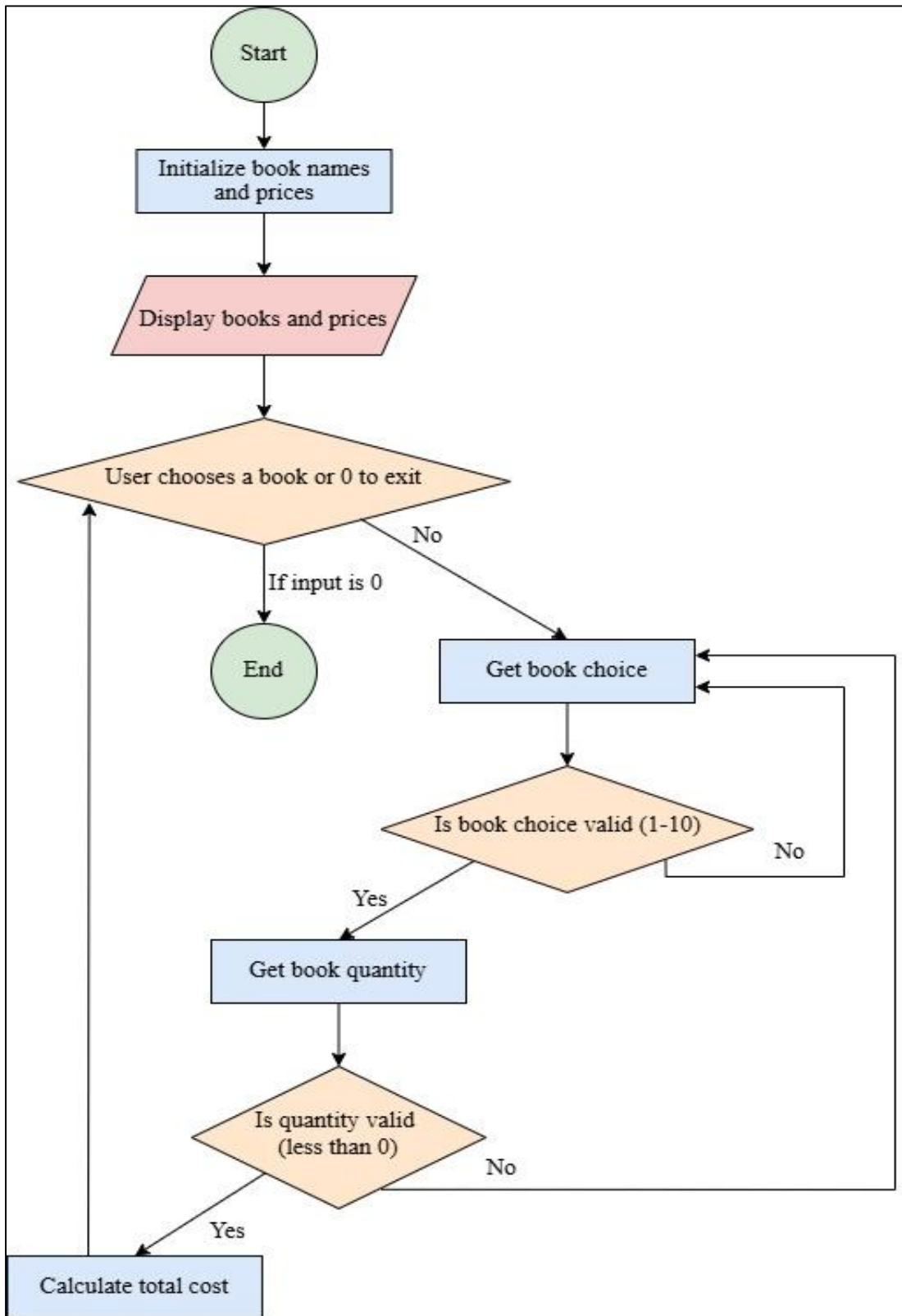


Fig 3.1 Flowchart for task 3

SOURCE CODE

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */
import java.util.Scanner;
public class Book_Shop {
    public static void main(String[] args) {
        // Books and their prices
        String[] books = { "Java Programming",
                           "Python Programming",
                           "C++ Programming",
                           "Data Structures",
                           "Algorithms",
                           "Web Development",
                           "Database Systems",
                           "Operating Systems",
                           "Software Engineering",
                           "Computer Networks" };

        double[] prices = {29.99, 24.99, 27.99, 32.50, 35.00,
                           30.00, 28.99, 33.00, 40.00,
                           31.50};

        Scanner input = new Scanner(System.in);
```



```

        // total cost of books
        double total = 0;

        System.out.println("Welcome to the Online Book
Shop!");

        // display books and prices
        booknames(books, prices);

        while (true) {
            // input books to buy
            int choice = bookchoice(input, books.length);
            // exit loop if input is 0
            if (choice == 0) {
                break;
            }
            // input quantity of chosen book
            int quantity = bookquantity(input, books[choice -
1]);

            // if quantity is greater than 0 calculate cost
            if (quantity <= 0) {
                continue;
            }
            // Total cost
            total += prices[choice - 1] * quantity;
            // Display the chosen the book
            System.out.println("Added " + quantity + "
copy/copies of '" + books[choice - 1] + "' to your cart.");

```

```

    }

    // Display total cost
    System.out.println("\nYour total amount is: " +
total);

    System.out.println("\nThank you for purchasing from
us. Visit again!");
}

// Method to display books and prices
public static void booknames(String[] books, double[]
prices) {
    System.out.println("\nBooks to buy: ");
    for (int i = 0; i < books.length; i++) {
        System.out.println((i + 1) + ". " + books[i] + "
- " + prices[i]);
    }
}

// Method to ask user to choose a book
public static int bookchoice(Scanner input, int
numberofbooks) {
    System.out.print("\nChoose a book to buy (or 0 to
exit): ");
    int choice = input.nextInt();
    // Check if chosen book is from the list
    if (choice < 0 || choice > numberofbooks) {
        System.out.println("\nWrong input. Choose a valid
book number.");
        return bookchoice(input, numberofbooks); // ask
for input again
    }
}

```

```
        }  
        return choice;  
    }  
  
    // Method to ask user for the quantity of chosen book  
    public static int bookquantity(Scanner input, String  
bookname) {  
        System.out.print("Enter the quantity of '" + bookname  
+ "': ");  
        int quantity = input.nextInt();  
        // if quantity is less than 0 ask for input again  
        if (quantity <= 0) {  
            System.out.println("Quantity should be more than  
0. Try again.");  
            return bookquantity(input, bookname);  
        }  
        return quantity;  
    }  
}
```

SCREENSHOT OF THE CODE

The program is written in Apache NetBeans IDE 23

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */
import java.util.Scanner;
public class Book_Shop {
    public static void main(String[] args) {
        // Books and their prices
        String[] books = { "Java Programming",
                           "Python Programming",
                           "C++ Programming",
                           "Data Structures",
                           "Algorithms",
                           "Web Development",
                           "Database Systems",
                           "Operating Systems",
                           "Software Engineering",
                           "Computer Networks" };

        double[] prices = {29.99, 24.99, 27.99, 32.50, 35.00,
                           30.00, 28.99, 33.00, 40.00, 31.50};

        Scanner input = new Scanner(System.in);
```

Fig 3.2 Source code for task 3

```
// total cost of books
double total = 0;

System.out.println("Welcome to the Online Book Shop!");

// display books and prices
booknames(books, prices);

while (true) {
    // input books to buy
    int choice = bookchoice(input, books.length);
    // exit loop if input is 0
    if (choice == 0) {
        break;
    }
    // input quantity of chosen book
    int quantity = bookquantity(input, books[choice - 1]);
    // if quantity is greater than 0 calculate cost
    if (quantity <= 0) {
        continue;
    }
    // Total cost
    total += prices[choice - 1] * quantity;
    // Display the chosen the book
    System.out.println("Added " + quantity + " copy/copies of '" +
                       books[choice - 1] + "' to your cart.");
```

Fig 3.3 Source code for task 3

```

    }
    // Display total cost
    System.out.println("\nYour total amount is: " + total);
    System.out.println("\nThank you for purchasing from us. Visit again!");
}

// Method to display books and prices
public static void booknames(String[] books, double[] prices) {
    System.out.println("\nBooks to buy: ");
    for (int i = 0; i < books.length; i++) {
        System.out.println((i + 1) + ". " + books[i] + " - " + prices[i]);
    }
}

// Method to ask user to choose a book
public static int bookchoice(Scanner input, int numberofbooks) {
    System.out.print("\nChoose a book to buy (or 0 to exit): ");
    int choice = input.nextInt();
    // Check if chosen book is from the list
    if (choice < 0 || choice > numberofbooks) {
        System.out.println("\nWrong input. Choose a valid book number.");
        return bookchoice(input, numberofbooks); // ask for input again
    }
    return choice;
}

```

Fig 3.4 Source code for task 3

```

// Method to ask user for the quantity of chosen book
public static int bookquantity(Scanner input, String bookname) {
    System.out.print("Enter the quantity of '" + bookname + "': ");
    int quantity = input.nextInt();
    // if quantity is less than 0 ask for input again
    if (quantity <= 0) {
        System.out.println("Quantity should be more than 0. Try again.");
        return bookquantity(input, bookname);
    }
    return quantity;
}
}

```

Fig 3.5 Source code for task 3

OUTPUT OF THE CODE

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---  
Welcome to the Online Book Shop!  
  
Books to buy:  
1. Java Programming - 29.99  
2. Python Programming - 24.99  
3. C++ Programming - 27.99  
4. Data Structures - 32.5  
5. Algorithms - 35.0  
6. Web Development - 30.0  
7. Database Systems - 28.99  
8. Operating Systems - 33.0  
9. Software Engineering - 40.0  
10. Computer Networks - 31.5  
  
Choose a book to buy (or 0 to exit): 1  
Enter the quantity of 'Java Programming': 3  
Added 3 copy/copies of 'Java Programming' to your cart.  
  
Choose a book to buy (or 0 to exit): 6  
Enter the quantity of 'Web Development': 2  
Added 2 copy/copies of 'Web Development' to your cart.  
  
Choose a book to buy (or 0 to exit): 10  
Enter the quantity of 'Computer Networks': 4  
Added 4 copy/copies of 'Computer Networks' to your cart.  
  
Choose a book to buy (or 0 to exit): 0  
  
Your total amount is: 275.97  
  
Thank you for purchasing from us. Visit again!  
-----  
BUILD SUCCESS  
-----  
Total time: 57.980 s  
Finished at: 2024-12-14T12:52:14+04:00
```

Fig 3.6 Output for task 3

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---
Welcome to the Online Book Shop!

Books to buy:
1. Java Programming - 29.99
2. Python Programming - 24.99
3. C++ Programming - 27.99
4. Data Structures - 32.5
5. Algorithms - 35.0
6. Web Development - 30.0
7. Database Systems - 28.99
8. Operating Systems - 33.0
9. Software Engineering - 40.0
10. Computer Networks - 31.5

Choose a book to buy (or 0 to exit): 12

Wrong input. Choose a valid book number.

Choose a book to buy (or 0 to exit): 5
Enter the quantity of 'Algorithms': 0
Quantity should be more than 0. Try again.
Enter the quantity of 'Algorithms': -1
Quantity should be more than 0. Try again.
Enter the quantity of 'Algorithms': 2
Added 2 copy/copies of 'Algorithms' to your cart.

Choose a book to buy (or 0 to exit): 0

Your total amount is: 70.0

Thank you for purchasing from us. Visit again!
-----
BUILD SUCCESS
-----
Total time: 02:26 min
Finished at: 2024-12-19T21:45:53+04:00
```

Fig 3.7 Output for task 3

TASK 4 - Favorite Destination

Write a program `Favorite_Destination` that computes the tally in a survey and announces the most wished city to visit by the tourists. Since the choices are write-in, there is no pre-determined list of cities that participate in the survey. Whichever city appears the most in the survey is declared the FAVOURITE CITY. The user enters the individual choice, one choice per line, and ends entering with typing QUIT/EXIT.

To compute the tally, the program uses two arrays, a `String [ ]` variable (`destination`), and an `int [ ]` variable (`count`). Upon receiving a single vote, the program checks if the entered city appears in `destination`, and if it does, the program adds 1 to the value of the element in `count`. If the city does not appear in `destination`, the program extends both arrays by one element, stores the city in `destination` at the last position and store 1 in `count` at the last position. In this manner, the two arrays will have the same lengths. The initial length is 0 for both arrays.

Hint: You can use either `Array` or `ArrayList`.

Below is an example of how the program may run:

```
-----< com.mycompany:election >-----
[-] Building election 1.0-SNAPSHOT
-----[ jar ]-----

[-] --- exec-maven-plugin:3.0.0:exec (default-cli)
Enter the city you love to visit
London
Dubai
Newyork
London
London
Singapore
Dubai
London
q
4 people have voted for London
2 people have voted for Dubai
1 people have voted for Newyork
1 people have voted for Singapore
LONDON is declared as the choice of tourists with 4 votes

BUILD SUCCESS

Total time: 39.765 s
Finished at: 2022-04-23T02:27:02-07:00
-----
```


EXPLANATION OF THE CODE

This program Favorite destination allows users to vote for cities they wish to visit and declares the most favorite city. This process continues until the user types QUIT or EXIT. It uses array lists to store city names and vote. As each city name is typed, the program checks if it's already there in the list. If it exists the vote count is incremented by one. If it's a new city it is added to the list.

Once all votes are recorded, the program displays votes for each city. It then determines the city with the highest number of votes and declares the favorite destination.

The following methods are implemented in the program to simplify its functionality

Methods-

- **Votes()** - collects user input and manages votes. It keeps running until the user types quit or exit. If the city exists in the list, its vote is incremented. If not it is added to the list.
- **Result()** - this method checks the destination and counts lists to display total votes for each city. It displays each city name and the votes it has received.
- **FavCity()** - this method displays the city with the highest votes. It checks the counts list and the city with maximum votes is printed along with city name from destinations list.
- **FavcityResult()** - this method takes input of favorite city and its vote count and displays the result. It prints the name of the most voted city and the number of votes it has received. If the user inputs quit or exit without entering any city name it prints no cities were entered.

FLOWCHART

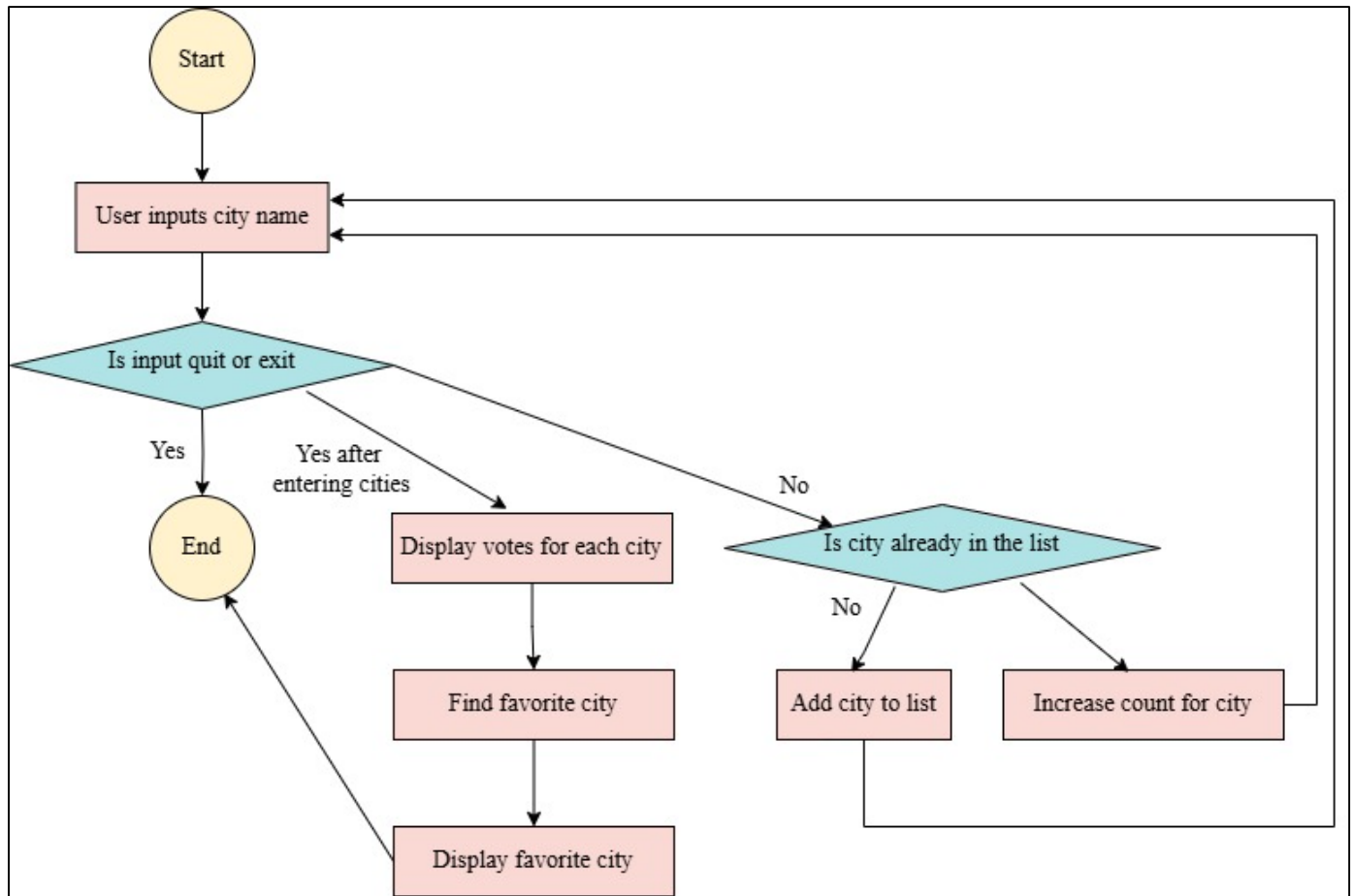


Fig 4.1 Flowchart for task 4

SOURCE CODE

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */
import java.util.ArrayList;
import java.util.Scanner;
public class Favorite_Destination {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        // create list to store city names
        ArrayList<String> destinations = new
ArrayList<String>();
        // create list to store votes
        ArrayList<Integer> counts = new ArrayList<Integer>();

        // ask user to enter their favorite cities
        System.out.println("Enter the city you would love to
visit "
            + "(type QUIT/EXIT to finish):");

        // count votes for each city
        votes(input, destinations, counts);

        // display votes for each city
        result(destinations, counts);
    }
}
```

```

        // find city highest vote
        String favoriteCity = FavCity(destinations, counts);

        // favorite city result
        FavcityResult(favoriteCity, counts);
    }

    // Method to count votes for each city
    public static void votes(Scanner scanner,
        ArrayList<String> destinations,
        ArrayList<Integer> counts) {
        while (true) {
            String input = scanner.nextLine();

            // check if input is quit or exit
            if(input.equalsIgnoreCase("QUIT") ||
input.equalsIgnoreCase("EXIT"))
            {
                break; // exit if input is quit or exit
            }

            // Check if the city is already exists
            boolean found = false;
            for (int i = 0; i < destinations.size(); i++) {
                if
(destinations.get(i).equalsIgnoreCase(input)) {
                    // if yes increment the vote
                    counts.set(i, counts.get(i) + 1);
                }
            }
        }
    }
}

```

```

        found = true;
        break;
    }
}

// If city is not found add to list
if (!found) {
    destinations.add(input);
    counts.add(1);
}
}

// Method to display votes for each city
public static void result(ArrayList<String> destinations,
    ArrayList<Integer> counts) {
    System.out.println("\nVotes for each city:");
    for (int i = 0; i < destinations.size(); i++) {
        // print each city and its votes
        System.out.println(destinations.get(i) + ": " +
counts.get(i)
            + " votes");
    }
}

// Method to find the city with maximum votes
public static String FavCity(ArrayList<String>
destinations,

```

```

        ArrayList<Integer> counts) {
// variable to store city
String favoriteCity = "";
// variable to store votes
int maxCount = 0;

// check through the lists to find city with maximum
votes
for (int i = 0; i < counts.size(); i++) {
    if (counts.get(i) > maxCount) {
        maxCount = counts.get(i);
        // update the favourite city
        favoriteCity = destinations.get(i);
    }
}
return favoriteCity;
}

```

```

// Method to display the favorite city and its vote
public static void FavcityResult(String favoriteCity,
        ArrayList<Integer> counts) {
    if (!favoriteCity.isEmpty()) {
        // find city with maximum vote
        int maxCount = 0;
        for (int count : counts) {
            if (count > maxCount) {
                maxCount = count;
            }
        }
    }
}

```

```
        }
        // display the favorite city and its vote
        System.out.println("\nThe favorite city to visit
is: "
                           + favoriteCity + " with " + maxCount + "
votes.");
    } else {
        // if no cities were entered
        System.out.println("No cities were entered.");
    }
}
}
```

SCREENSHOT OF THE CODE

The program is written in Apache NetBeans IDE 23

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */
import java.util.ArrayList;
import java.util.Scanner;
public class Favorite_Destination {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        // create list to store city names
        ArrayList<String> destinations = new ArrayList<String>();
        // create list to store votes
        ArrayList<Integer> counts = new ArrayList<Integer>();

        // ask user to enter their favorite cities
        System.out.println("Enter the city you would love to visit "
            + "(type QUIT/EXIT to finish):");

        // count votes for each city
        votes(input, destinations, counts);

        // display votes for each city
        result(destinations, counts);
    }
}
```

Fig 4.2 Source code for task 4

```
// find city highest vote
String favoriteCity = FavCity(destinations, counts);

// favorite city result
FavcityResult(favoriteCity, counts);
}

// Method to count votes for each city
public static void votes(Scanner scanner, ArrayList<String> destinations,
    ArrayList<Integer> counts) {
    while (true) {
        String input = scanner.nextLine();

        // check if input is quit or exit
        if(input.equalsIgnoreCase("QUIT") || input.equalsIgnoreCase("EXIT"))
        {
            break; // exit if input is quit or exit
        }

        // Check if the city is already exists
        boolean found = false;
        for (int i = 0; i < destinations.size(); i++) {
            if (destinations.get(i).equalsIgnoreCase(input)) {
                // if yes increment the vote
                counts.set(i, counts.get(i) + 1);
            }
        }
    }
}
```

Fig 4.3 Source code for task 4


```

        counts.set(i, counts.get(i) + 1);
        found = true;
        break;
    }
}

// If city is not found add to list
if (!found) {
    destinations.add(input);
    counts.add(1);
}
}

// Method to display votes for each city
public static void result(ArrayList<String> destinations,
    ArrayList<Integer> counts) {
    System.out.println("\nVotes for each city:");
    for (int i = 0; i < destinations.size(); i++) {
        // print each city and its votes
        System.out.println(destinations.get(i) + ": " + counts.get(i)
            + " votes");
    }
}

```

Fig 4.4 Source code for task 4

```

// Method to find the city with maximum votes
public static String FavCity(ArrayList<String> destinations,
    ArrayList<Integer> counts) {
    // variable to store city
    String favoriteCity = "";
    // variable to store votes
    int maxCount = 0;

    // check through the lists to find city with maximum votes
    for (int i = 0; i < counts.size(); i++) {
        if (counts.get(i) > maxCount) {
            maxCount = counts.get(i);
            // update the favourite city
            favoriteCity = destinations.get(i);
        }
    }
    return favoriteCity;
}

// Method to display the favorite city and its vote
public static void FavCityResult(String favoriteCity,
    ArrayList<Integer> counts) {
    if (!favoriteCity.isEmpty()) {
        // find city with maximum vote
        int maxCount = 0;
    }
}

```

Fig 4.5 Source code for task 4

```

        for (int count : counts) {
            if (count > maxCount) {
                maxCount = count;
            }
        }
        // display the favorite city and its vote
        System.out.println("\nThe favorite city to visit is: "
            + favoriteCity + " with " + maxCount + " votes.");
    } else {
        // if no cities were entered
        System.out.println("No cities were entered.");
    }
}
}

```

Fig 4.6 Source code for task 4

OUTPUT OF THE CODE

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---
Enter the city you would love to visit (type QUIT/EXIT to finish):
London
Dubai
New york
Sydney
Dubai
London
Paris
Dubai
Tokyo
exit

Votes for each city:
London: 2 votes
Dubai: 3 votes
New york: 1 votes
Sydney: 1 votes
Paris: 1 votes
Tokyo: 1 votes

The favorite city to visit is: Dubai with 3 votes.
-----
BUILD SUCCESS
-----
Total time: 01:52 min
Finished at: 2024-12-14T19:41:31+04:00
-----
```

Fig 4.7 Output for task 4

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---
Enter the city you would love to visit (type QUIT/EXIT to finish):
quit

Votes for each city:
No cities were entered.
-----
BUILD SUCCESS
-----
Total time: 8.141 s
Finished at: 2024-12-19T21:56:40+04:00
-----
```

Fig 4.8 Output for task 4

TASK 5 - Account Management

Task 5 is on Object Oriented

Programming. Design a class named Account that contains:

- ☐ A private int data field named `ac_id` for the account (default 1).
- ☐ A private double data field named `ac_balance` for the account (default 0.0).
- ☐ A private double data field named `annualInterestRate` that stores the current interest rate (default 0.0). Assume all accounts have the same interest rate.
- ☐ A private Date data field named `ac_dateCreated` that stores the date when the account was created.
- ☐ A no-arg constructor that creates a default account.
- ☐ A constructor that creates an account with the specified id and initial balance.
- ☐ The accessor and mutator methods for `ac_id`, `ac_balance`, and `annualInterestRate`.
- ☐ The accessor method for `ac_dateCreated`.
- ☐ A method named `getMonthlyInterestRate()` that returns the monthly interest rate.
- ☐ A method named `withdraw` that withdraws a specified amount from the account.
- ☐ A method named `deposit` that deposits a specified amount to the account.
- ☐ Draw the UML diagram for the class.
- ☐ Implement the class. Write a Java program that creates an Account object with an account ID of 1001, a balance of \$30,000, and an annual interest rate of 5.5%. Use the `withdraw` method to withdraw \$3,500, use the `deposit` method to deposit \$4,500, and print the balance, the monthly interest, and the date when this account was created.

EXPLANATION OF THE CODE

This program is an object oriented program to manage a bank account using Account class. The class includes account id, balance, annual interest rate and date to store account creation date.

Methods in the class include –

- **Accessors and Mutators:**
 - **getAc_id** and **setAc_id** for the account ID.
 - **getAc_balance** and **setAc_balance** for the account balance.
 - **getAnnualInterestRate** and **setAnnualInterestRate** for the static annual interest rate.
 - **getAc_dateCreated** for the account creation date.
- **Calculations**
 - **Withdraw** - This method deducts the specified amount from balance.
 - **Deposit** – This method adds the specified amount to the balance
 - **getMonthlyInterestRate()** - This method divides the annual interest rate by 12 to calculate the monthly interest.

The main method creates an Account object with account id as 1001, a balance of 30,000 and an annual interest rate of 5.5%. It performs a withdrawal of 3,500 and a deposit of 4,500 then prints the account id, current balance, monthly interest rate and account creation date

UML DIAGRAM

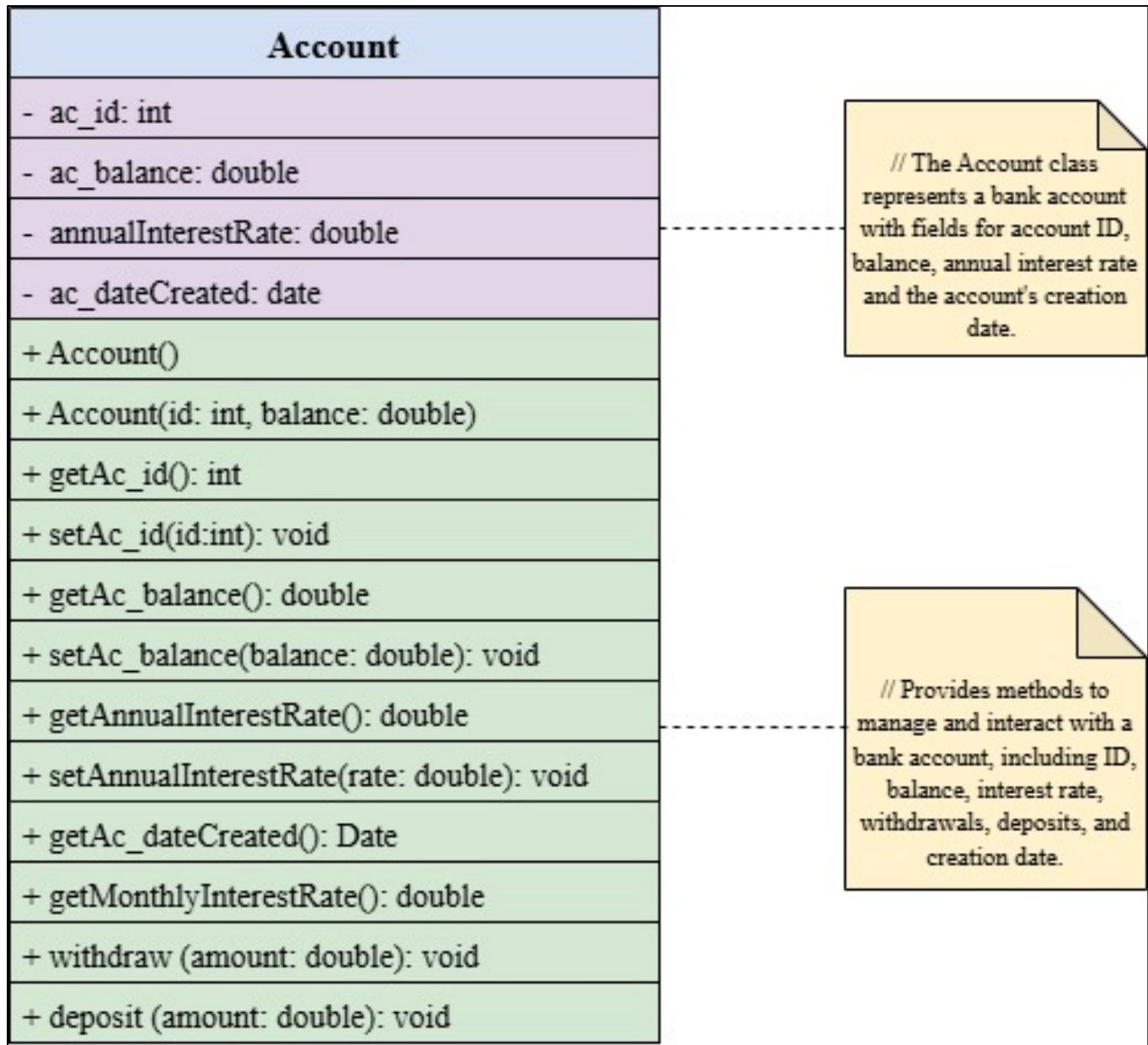


Fig 5.1 UML diagram for task 5

SOURCE CODE

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */
import java.util.Date;
// class to display bank account
class Account {
    private int ac_id=1; // default account id
    private double ac_balance=0.0; // default account balance
    private static double annualInterestRate=0.0; // annual
    interest rate
    private Date ac_dateCreated; // account creation date

    public Account(){
        // set account creation date to current date
        this.ac_dateCreated=new Date();
    }

    // constructor that initializes account id, balance, date
    public Account(int id, double balance){
        this.ac_id=id;
        this.ac_balance=balance;
        this.ac_dateCreated=new Date();
    }
}
```

```
// getter for account id
public int getAc_id(){
    return ac_id;
}

// setter for account id
public void setAc_id(int id){
    this.ac_id=id;
}

// getter for account balance
public double getAc_balance(){
    return ac_balance;
}

// setter for accouunt balance
public void setAc_balance(double balance){
    this.ac_balance=balance;
}

// getter for annual interest rate
public static double getAnnualInterestRate(){
    return annualInterestRate;
}

// setter for annual interest rate
public static void setAnnualInterestRate(double rate){
```



```
        annualInterestRate=rate;
    }

    // getter for account creation date
    public Date getAc_dateCreated(){
        return ac_dateCreated;
    }

    // method to calculate monthly interest rate
    public double getMonthlyInterestRate(){
        return annualInterestRate/12;
    }

    // method to withdraw specified amount
    public void withdraw(double amount){
        // reduce the amount from balance
        ac_balance-=amount;
    }

    // method to deposit specified amount
    public void deposit(double amount){
        // add the amount to balance
        ac_balance+=amount;
    }
}

// Main class to use Account class
public class BankAccount{
    public static void main(String[] args) {
```

```

        // create account with id and balance
        Account account = new Account(1001,30000);

        // set annual interest rate
        Account.setAnnualInterestRate(5.5);

        // print account details
        System.out.println("Account ID: " +
account.getAc_id());
        System.out.println("Balance: $" +
account.getAc_balance());
        System.out.println("Monthly Interest Rate: " +
            account.getMonthlyInterestRate() + "%");
        System.out.println("Date Created: " +
account.getAc_dateCreated());

        // withdraw 3500 from account
        account.withdraw(3500);
        System.out.println("After withdrawing $3500, balance
is "
            + account.getAc_balance());

        // deposit 4500 to account
        account.deposit(4500);
        System.out.println("After depositing $4500, balance
is "
            + account.getAc_balance());
    }
}

```

SCREENSHOT OF THE CODE

The program is written in Apache NetBeans IDE 23

```
/**
 * @author Nasiha
 * Student Id - 33150863
 * Batch - 2
 */
import java.util.Date;
// class to display bank account
class Account {
    private int ac_id=1; // default account id
    private double ac_balance=0.0; // default account balance
    private static double annualInterestRate=0.0; // annual interest rate
    private Date ac_dateCreated; // account creation date

    public Account(){
        // set account creation date to current date
        this.ac_dateCreated=new Date();
    }

    // constructor that initializes account id, balance, date
    public Account(int id, double balance){
        this.ac_id=id;
        this.ac_balance=balance;
        this.ac_dateCreated=new Date();
    }
}
```

Fig 5.2 Source code for task 5

```
// getter for account id
public int getAc_id(){
    return ac_id;
}

// setter for account id
public void setAc_id(int id){
    this.ac_id=id;
}

// getter for account balance
public double getAc_balance(){
    return ac_balance;
}

// setter for account balance
public void setAc_balance(double balance){
    this.ac_balance=balance;
}

// getter for annual interest rate
public static double getAnnualInterestRate(){
    return annualInterestRate;
}
```

Fig 5.3 Source code for task 5

```

// setter for annual interest rate
public static void setAnnualInterestRate(double rate){
    annualInterestRate=rate;
}

// getter for account creation date
public Date getAc_dateCreated(){
    return ac_dateCreated;
}

// method to calculate monthly interest rate
public double getMonthlyInterestRate(){
    return annualInterestRate/12;
}

// method to withdraw specified amount
public void withdraw(double amount){
    // reduce the amount from balance
    ac_balance-=amount;
}

// method to deposit specified amount
public void deposit(double amount){
    // add the amount to balance
    ac_balance+=amount;
}

```

Fig 5.4 Source code for task 5

```

}
}
// Main class to use Account class
public class BankAccount{
    public static void main(String[] args) {
        // create account with id and balance
        Account account = new Account(1001,30000);

        // set annual interest rate
        Account.setAnnualInterestRate(5.5);

        // print account details
        System.out.println("Account ID: " + account.getAc_id());
        System.out.println("Balance: $" + account.getAc_balance());
        System.out.println("Monthly Interest Rate: " +
            account.getMonthlyInterestRate() + "%");
        System.out.println("Date Created: " + account.getAc_dateCreated());

        // withdraw 3500 from account
        account.withdraw(3500);
        System.out.println("After withdrawing $3500, balance is "
            + account.getAc_balance());

        // deposit 4500 to account
        account.deposit(4500);
        System.out.println("After depositing $4500, balance is "
            + account.getAc_balance());
    }
}

```

Fig 5.5 Source code for task 5

OUTPUT OF THE CODE

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---  
Account ID: 1001  
Balance: $31000.0  
Monthly Interest Rate: 0.4583333333333333%  
Date Created: Sun Dec 15 15:24:15 GMT+04:00 2024  
-----  
BUILD SUCCESS  
-----  
Total time: 2.406 s  
Finished at: 2024-12-15T15:24:15+04:00  
-----
```

Fig 5.6 Output for task 5

```
--- exec:3.1.0:exec (default-cli) @ Assignment ---  
Account ID: 1001  
Balance: $30000.0  
Monthly Interest Rate: 0.4583333333333333%  
Date Created: Thu Dec 19 22:12:50 GMT+04:00 2024  
After withdrawing $3500, balance is 26500.0  
After depositing $4500, balance is 31000.0  
-----  
BUILD SUCCESS  
-----  
Total time: 1.981 s  
Finished at: 2024-12-19T22:12:50+04:00
```

Fig 5.7 Output for task 5